Instituto Tecnológico Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

ESTRUCTURA DE DATOS

VAZQUES HERRERA VICTOR HUGO

UNIDAD 3
TAREA 2

PRESENTA

257O00081
Brayan Mejia Mora

18 DE OCTUBRE DEL 2026

Práctica de Programación en Java: Arquitectura Modular, Colas (Estática, Circular y Dinámica)

Cola Estática Lineal

Los siguientes fragmentos son clave porque concentran la lógica que diferencia a esta cola de las demás: la condición de cola llena basada únicamente en **fin**, y el avance lineal de **frente** sin reutilización de espacio.

```
================================================================================
public void enqueue(int valor) {
    if (fin == capacidad - 1) {
        System.out.println("Error: La cola esta llena.");
    } else {
        fin++;
        arreglo[fin] = valor;
        System.out.println("Entro a la fila: " + valor);
    }
}

public void dequeue() {
    if (frente > fin) {
        System.out.println("Cola esta vacia!");
    } else {
        System.out.println("Atendido y fuera: " + arreglo[frente]);
        frente++;
    }
}
================================================================================
```

¿Por qué son clave?

El método **enqueue** revisa solo si **fin** llegó al final del arreglo, sin considerar el espacio libre al inicio.

El método **dequeue** solo avanza **frente** sin liberar ni reutilizar la posición.

En conjunto, evidencian la limitación principal de esta implementación: la falsa cola llena.

Captura de consola

```
>> Insertando 10, 20 y 30...
Entro a la fila: 10
Entro a la fila: 20
Entro a la fila: 30
Estado actual: Fila actual: 10 20 30

>> Atendiendo un elemento...
Atendido y fuera: 10
Estado actual: Fila actual: 20 30

>> Intentando insertar 40 (debería fallar)...
Error: La cola esta llena.
Estado actual: Fila actual: 20 30

>> Atendiendo los elementos restantes...
Atendido y fuera: 20
Atendido y fuera: 30
Estado final: La cola esta vacia!
```

Cola Estática Circular

Los siguientes fragmentos son clave porque muestran las dos modificaciones esenciales respecto a la cola lineal: el avance circular con **%** y el control del estado mediante la variable **tamaño**.

```
================================================================================
public void enqueue(int valor) {
    if (isFull()) {
        System.out.println("Error: La cola esta llena.");
        return;
    }

    fin = (fin + 1) % capacidad;
    arreglo[fin] = valor;
    tamaño++;

    System.out.println("Entro a la fila: " + valor);
}

public int dequeue() {
    if (isEmpty()) {
        System.out.println("Error: La cola esta vacia!");
        return -1;
    }

    int atendido = arreglo[frente];
    frente = (frente + 1) % capacidad;
    tamaño--;

    System.out.println("Atendido y fuera: " + atendido);
    return atendido;
}
================================================================================
```

¿Por qué son clave?

La operación **fin = (fin + 1) % capacidad** permite que el índice regrese a 0 cuando llega al final, reutilizando espacio.

La operación **frente = (frente + 1) % capacidad** hace lo mismo con el frente.

La variable **tamaño** resuelve la ambigüedad entre cola llena y cola vacía, algo imposible de distinguir solo con **frente** y **fin** en una estructura circular.

Captura de consola

```
Entro a la fila: 50
Estado actual: Fila actual: 10 20 30 40 50

>> Atendiendo dos elementos...
Atendido y fuera: 10
Atendido y fuera: 20
Estado actual: Fila actual: 30 40 50

>> Insertando 60 y 70 (reutilizando espacio)...
Entro a la fila: 60
Entro a la fila: 70
Estado actual: Fila actual: 30 40 50 60 70

>> Consultando el frente...
Valor al frente de la cola: 30

>> Vaciando la cola por completo...
Atendido y fuera: 30
Atendido y fuera: 40
Atendido y fuera: 50
Atendido y fuera: 60
Atendido y fuera: 70

Estado final: La cola esta vacia!
¿Está vacía? true
```

Cola Dinámica

Los siguientes fragmentos son clave porque muestran cómo se gestionan las referencias **frente** y **fin** sin arreglo, y en particular el caso especial de dejar **fin = null** cuando la cola queda vacía.

```
================================================================================
public void encolar(int x) {
    Nodo nuevo = new Nodo(x);
    if (estaVacia()) {
        frente = nuevo;
        fin = nuevo;
    } else {
        fin.siguiente = nuevo;
        fin = nuevo;
    }
}

public int desencolar() {
    if (estaVacia()) {
        System.out.println("La cola esta vacia!");
        return -1;
    }

    int valor = frente.dato;
    frente = frente.siguiente;

    if (frente == null) {
        fin = null;
    }

    return valor;
}
================================================================================
```

¿Por qué son clave?

El método **encolar** resuelve el caso de cola vacía apuntando **frente** y **fin** al mismo nodo, y el caso general enlazando al final.

El método **desencolar** recupera el dato antes de perder la referencia y ajusta **fin** a **null** cuando la cola queda vacía.

Sin ese ajuste final, **fin** quedaría apuntando a un nodo ya eliminado y **estaVacia()** seguiría funcionando, pero la coherencia estructural se rompería.

Captura de consola

```
=== ESTADO INICIAL ===
¿Esta vacia? true
Pila Dinamica (Cima → Fondo): [VACIA]

=== PEEK CON PILA VACIA ===
Pila dinamica vacia! No hay cima que consultar.

=== POP CON PILA VACIA ===
Pila dinamica vacia! No se puede sacar.

=== INSERTANDO ELEMENTOS (PUSH) ===
Pila Dinamica (Cima → Fondo): 50 40 30 20 10

=== PEEK EN LA CIMA ===
Consultaste: 50

=== SACANDO ELEMENTOS (POP) ===
Sacaste de la pila: 50
Sacaste de la pila: 40
Pila Dinamica (Cima → Fondo): 30 20 10

=== VACIANDO LA PILA ===
Sacaste de la pila: 30
Sacaste de la pila: 20
Sacaste de la pila: 10

=== VERIFICANDO ESTADO FINAL ===
¿Esta vacia? true
Pila Dinamica (Cima → Fondo): [VACIA]

=== POP EN PILA VACIA ===
Pila dinamica vacia! No se puede sacar.
```

Conclusión comparativa

Comportamiento en memoria

Aspecto	Estática Lineal	Estática Circular	Dinámica
Estructura base	Arreglo de tamaño fijo	Arreglo de tamaño fijo	Nodos enlazados
Memoria reservada	Completa desde el inicio	Completa desde el inicio	Solo la usada
Desperdicio de memoria	Alto (posiciones liberadas no se reutilizan)	Nulo (posiciones se reutilizan)	Nulo (nodos se liberan al desencolar)
Fragmentación	No aplica	No aplica	Puede haber fragmentación por nodos dispersos
Overhead por elemento	Bajo (solo el dato)	Bajo (solo el dato)	Mayor (dato + referencia siguiente)

Limitaciones de capacidad

Aspecto	Estática Lineal	Estática Circular	Dinámica
Capacidad	Fija, definida al crear	Fija, definida al crear	Ilimitada (hasta agotar memoria)
Cola llena	Se reporta al llegar fin al final (capacidad -1)	Se reporta solo cuando tamaño == capacidad	Nunca (salvo OutOfMemoryError)
Falsa cola llena	Sí ocurre	No ocurre	No aplica

Ventajas operativas

Aspecto	Estática Lineal	Estática Circular	Dinámica
enqueue() / encolar()	O(1)	O(1)	O(1)
dequeue() / desencolar()	O(1)	O(1)	O(1)
isEmpty() / estaVacia()	Indirecto (frente > fin)	Directo (tamaño == 0)	Directo (frente == null)
isFull() / estaLlena()	Indirecto (fin == capacidad - 1)	Directo (tamaño == capacidad)	No aplica
Reutiliza espacio	No	Sí	Sí
Complejidad de implementación	Baja	Media (uso de % y tamaño)	Media (manejo de referencias)

La cola estática lineal es la más simple de implementar, pero desperdicia memoria y presenta el problema de la falsa cola llena. Es útil únicamente con fines didácticos.

La cola estática circular corrige el problema anterior reutilizando el espacio liberado mediante índices circulares. Es eficiente cuando se conoce de antemano la capacidad máxima y se busca un uso óptimo de memoria.

La cola dinámica elimina la restricción de capacidad y se adapta al crecimiento o decrecimiento de la demanda. Su costo es un mayor overhead (costo general) por nodo y una implementación ligeramente más compleja.

La elección entre las tres depende del contexto. Si la capacidad es conocida y fija, la circular es la mejor opción. Si la capacidad es variable o impredecible, la dinámica es la más adecuada. La lineal queda como referencia conceptual para comprender por qué surgieron las otras dos.